

Mission 005 Architecture Packet

1) Canonical system design / trust boundaries

Target architecture

- **Canonical source of truth:** a single Neon Postgres database provisioned through the user's Vercel setup.
- **Write path:** agents write only through the backend API.
- **Read path:** the app reads only through backend read APIs that query Neon.
- **UI:** stays read-only for end users; it never writes to the database directly.
- **JSON:** remains optional only as an export/debug/bootstrap artifact; it is **not** the primary truth path.

Boundaries

- **Trusted:**
 - Neon database state.
 - Backend API validation and persistence logic.
 - Read API response shaping.
- **Untrusted / secondary:**
 - Any JSON file or remote JSON feed.
 - Client-side state for anything beyond presentation.
 - Agent payloads before server validation.

Current repo files this replaces as the truth path

- ``src/lib/dashboard-source.ts`` currently loads JSON from file/URL/default remote GitHub JSON.
- ``src/lib/dashboard-service.ts`` currently builds responses from the JSON source.
- ``src/lib/dashboard-data.ts`` currently normalizes cards/timeline/freshness from that source.
- ``src/app/api/dashboard/route.ts`` and ``src/app/api/missions/latest/route.ts`` currently expose read-only GET routes.
- ``src/lib/dashboard-types.ts`` currently defines the read contracts.
- ``src/lib/dashboard-live-source.json`` is the embedded source artifact.

What should stop relying on JSON as truth

- Production and preview reads should stop depending on ``src/lib/dashboard-live-source.json``.
- ``loadDashboardDataSource()`` should stop preferring JSON URLs/files for canonical runtime behavior.
- Dashboard and latest-mission APIs should stop using JSON-backed fixtures as their live data source.
- Tests should move to DB/fixture abstractions that mirror Neon rows rather than treating JSON as production truth.

2) Proposed database schema

Design principles

- Keep the schema small and explicit.
- Store immutable agent actions as events.
- Store one current mission snapshot for fast reads.
- Preserve ordering, freshness, and visibility metadata.
- Make all write operations idempotent.

Table: `missions`

Represents the current canonical state for each mission.

****Key fields****

- `id` UUID or text PK, stable mission identifier.
- `slug` text unique, human-readable mission key if needed.
- `title` text not null.
- `status` text not null, constrained to `running | completed | unknown`.
- `headline` text nullable, presentation-friendly summary.
- `detail` text nullable, longer presentation text.
- `actor\_name` text nullable.
- `actor\_role` text nullable.
- `latest\_action` text nullable.
- `started\_at` timestampz nullable.
- `updated\_at` timestampz not null, canonical last-change timestamp.
- `version` bigint not null default 0, monotonic mission version.
- `source\_label` text nullable, e.g. `neon-canonical-db`.
- `created\_at` timestampz not null default now().
- `ingested\_at` timestampz not null default now(), when the backend persisted the latest mutation.

****Indexes / constraints****

- `PRIMARY KEY (id)`.
- `UNIQUE (slug)` if slugs are used.
- Index on `updated\_at DESC`.
- Index on `status`.
- Optional partial index for `status = 'running'` if dashboard queries need it.

Table: `mission\_events`

Immutable append-only event log.

****Key fields****

- `id` UUID or text PK.
- `mission\_id` FK -> `missions.id`.
- `event\_type` text not null, e.g. `mission.created`, `mission.updated`, `mission.completed`.
- `actor\_name` text not null.
- `actor\_role` text nullable.
- `action` text not null.
- `detail` text nullable.
- `summary` text nullable.
- `event\_timestamp` timestamptz not null, the agent-supplied event time.
- `sequence\_index` bigint not null, ordering within a mission.
- `parallel\_group\_id` text nullable.
- `parallel\_order` integer nullable.
- `parallel\_size` integer nullable.
- `source\_label` text nullable.
- `request\_id` text nullable, idempotency key / trace key.
- `payload\_hash` text nullable, request integrity/debug.
- `created\_at` timestamptz not null default now().

****Indexes / constraints****

- `PRIMARY KEY (id)`.
- `UNIQUE (mission\_id, sequence\_index)`.
- `UNIQUE (request\_id)` when provided.
- Index on `(mission\_id, event\_timestamp ASC, sequence\_index ASC)` for timeline reads.
- Index on `(mission\_id, created\_at DESC)` for recent activity.

Optional table: `api\_write\_receipts`

Useful if replay detection and auditability need to be explicit.

****Key fields****

- `request\_id` text primary key.
- `mission\_id` FK nullable.
- `status` text not null (`accepted` | `rejected`).
- `reason` text nullable.

- `created\_at` timestampz not null default now().

Derived fields

- `missions.version` increments on every accepted write.
- `missions.updated\_at` equals the latest event timestamp or the server-accepted timestamp, whichever policy is chosen and then applied consistently.
- UI freshness fields are derived at read time, not stored as primary truth.

3) API contract for write endpoint(s) and read endpoint(s)

Write API v1

`POST /api/agent-events`

Validate the payload shape, derive a request hash server-side, and append a mission event while updating the mission snapshot in the same transaction. v1 does not require a write secret or signed request envelope.

****Request body****

```
{
  "missionId": "mission-005",
  "eventType": "mission.updated",
  "actorName": "Mira",
  "actorRole": "Backend Developer",
  "action": "persisted the first Neon-backed write",
  "detail": "The canonical DB now owns the new mission state.",
  "summary": "Backed the mission by Neon.",
  "eventTimestamp": "2026-05-27T17:24:05.000Z",
  "sequenceIndex": 8,
  "parallelGroupId": null,
  "parallelOrder": null,
  "parallelSize": null,
  "sourceLabel": "neon-canonical-db"
}
```

****Response 201****

```
{
  "ok": true,
  "requestId": "<server-derived request hash>",
  "mission": {
    "id": "mission-005",
    "title": "Canonical DB Architecture",
    "status": "running",
    "updatedAt": "2026-05-27T17:24:05.000Z",
    "version": 9
  },
  "event": {
    "id": "evt_...",
    "sequenceIndex": 8
  },
  "source": {
    "name": "neon-canonical-db",
  }
}
```

```
    "updatedAt": "2026-05-27T17:24:05.000Z"
  }
}
```

****Idempotent replay response 200****

- Same shape as success, but flagged as replayed/duplicate if the same raw request body is submitted again.

****Error responses****

- `400 Bad Request`: invalid JSON or invalid fields.
- `409 Conflict`: sequence collision, replay, or semantic conflict.
- `500 Internal Server Error`: persistence failure.
- `503 Service Unavailable`: DB unavailable in the current runtime.

Read APIs v1

`GET /api/dashboard`

Return dashboard state read from Neon.

****Response shape****

- Keep the existing `DashboardResponse` envelope from `src/lib/dashboard-types.ts`.
- `generatedAt` remains server-generated.
- `source.name` should identify the canonical DB-backed source, not JSON.
- `source.updatedAt` and `source.lagMs` should come from DB timestamps.

`GET /api/missions/latest`

Return the latest mission timeline read from Neon.

****Response shape****

- Keep the existing `MissionTimelineResponse` envelope.
- `mission` is the latest canonical mission snapshot.
- `events` are the ordered events from `mission\_events`.
- `lagMs`, `freshnessState`, `sourceStatus`, and `isStale` remain exposed.

Read error handling

- `200 OK` with `empty` freshness when there are no missions yet.
- `503 Service Unavailable` only if the DB is unreachable and no safe fallback is allowed in that environment.
- Prefer explicit empty-state responses over silent JSON fallback in production.

4) Auth/signing model recommendation for agent writes

Recommendation

Use a **simple server-validated write path** for v1.

Why this model

- Keeps the v1 cutover minimal.
- Avoids write-secret management for the first canonical DB pass.
- Lets the backend validate payload shape and persist canonical state directly.

Suggested write handling rules

- Validate the payload shape on the server before any DB write.
- Derive the write identity from the request body on the server.
- Preserve idempotent storage at the DB layer where available.
- Keep the frontend read-only.

5) Freshness / lag / updated-at semantics and UI behavior

Canonical semantics

- ``updated_at`` on the mission row is the canonical last-change timestamp.
- ``lagMs = generatedAt - updated_at`` on read.
- ``source.updatedAt`` should reflect the newest relevant DB timestamp for the payload.
- ``generatedAt`` is always the response generation timestamp.

Freshness states

Preserve the current visible states already used by the app contract in ``src/lib/dashboard-types.ts``:

- ``fresh``: recent data.
- ``partial``: some fields or events are incomplete.
- ``delayed``: fresh enough to show but lagging.
- ``stale``: too old to trust as current.
- ``empty``: no mission data yet.

Suggested thresholds

- ``fresh``: under 60s old.
- ``delayed``: 60s to under 15m old.
- ``stale``: 15m or older.
- ``partial``: any record/event missing required story fields.
- ``empty``: no rows.

UI display guidance

- Show `updatedAt` plainly in the timeline and dashboard cards.
- Show freshness badge or label near the latest mission and timeline header.
- Show lag explicitly instead of hiding it.
- The UI should continue to be read-only; no action buttons that mutate data.
- The UI should not infer state from JSON filenames, source URLs, or local fixtures.

6) Migration / backfill plan from the current JSON source to the DB

Source to migrate from

- The current repo-backed JSON source in `src/lib/dashboard-live-source.json`.
- Any live JSON feed currently selected by `src/lib/dashboard-source.ts`.

Backfill steps

1. Create the Neon schema and seed/migration scripts.
2. Normalize the current JSON records into `missions` and `mission\_events`.
3. Backfill once into Neon in an idempotent way.
4. Verify read APIs can render the same mission/timeline shape from Neon.
5. Switch production and preview reads to the DB path.
6. Keep JSON only as a debug/export/bootstrap artifact after cutover.

Cutover rules

- After backfill, all new writes go to the backend write API only.
- The app should stop depending on JSON replication as the main architecture.
- Any JSON export should be clearly labeled as secondary or diagnostic.
- Do not use the JSON file as the live production source once the DB path is live.

Backfill boundary

- Existing mission history can be imported once.
- New writes after cutover must not be dual-written to JSON as truth.
- If export is needed, generate it from DB state, not the other way around.

7) Rollout / phase boundaries

v1 scope

- Single canonical Neon database.
- Append-only agent write API.
- DB-backed `GET /api/dashboard`.
- DB-backed `GET /api/missions/latest`.
- Read-only UI.
- Freshness / lag visibility.
- One-time JSON backfill.
- Basic server-side validation for agent writes.

Later phases

- Realtime push or websocket updates.
- Outbox/queue if write throughput or fanout requires it.
- Multi-key rotation UI or admin tooling.
- Additional read endpoints such as mission history by id.
- Optional export jobs for JSON/debug artifacts.

Explicit non-goals for v1

- End-user mutation controls.
- Admin dashboards.
- A second dashboard view.
- Webhook-only ingestion if a direct simple API is simpler.
- Treating JSON as the primary path.

8) Risks and assumptions

Risks

- JSON and DB could drift during migration if cutover is not clean.
- Write ordering may be inconsistent if clients do not supply stable sequence metadata.
- Replay or duplicate submission risk without strict idempotency.
- Freshness calculations can become misleading if `updated\_at` semantics are inconsistent.
- Read latency may grow if dashboard queries are not indexed properly.

Assumptions

- Neon is available through the user's Vercel setup, per mission docs.

- The current UI is already read-only and should remain so.
- Mission 005 should keep the smallest architecture that gives a single source of truth and stable APIs.
- The current JSON source can be treated as seed/backfill material, not live truth.
- Mission documentation must exist in both markdown and PDF form.

9) Exact handoff instructions

Backend handoff

- Create the Neon schema with `missions` and `mission\_events`.
- Implement payload-shape validation and a simple canonical write path for `POST /api/agent-events`.
- Update `GET /api/dashboard` and `GET /api/missions/latest` to query Neon.
- Preserve the current response contracts from `src/lib/dashboard-types.ts`.
- Remove runtime dependence on `src/lib/dashboard-source.ts` JSON fallback in production.
- Add tests for payload validation, write persistence, ordering, and freshness.
- Produce/update a markdown handoff and the matching PDF artifact in `docs/missions/pdfs/`.

Frontend handoff

- Keep the UI read-only.
- Continue consuming the dashboard and latest-mission read APIs only.
- Display freshness, lag, and updated-at visibly.
- Do not infer truth from JSON filenames or source URLs.
- Keep the one-timeline experience unchanged in shape, only changing the backing source.
- Ensure empty, partial, delayed, and stale states remain legible.
- Produce/update a markdown handoff and the matching PDF artifact in `docs/missions/pdfs/`.

QA handoff

- Verify writes are accepted with valid payloads and rejected for invalid payload shape.
- Verify accepted writes appear in Neon and then in the read APIs.
- Verify dashboard and latest-mission responses remain schema-stable.
- Verify freshness / lag / updated-at visibility in production or preview.
- Verify the app still behaves read-only for end users.
- Verify JSON is no longer the source of truth after cutover.
- Verify migration/backfill preserved existing mission history.
- Produce/update a markdown handoff and the matching PDF artifact in `docs/missions/pdfs/`.

Repo-specific implementation note

The current live data path is still JSON-backed in:

- ``src/lib/dashboard-source.ts``
- ``src/lib/dashboard-service.ts``
- ``src/lib/dashboard-data.ts``
- ``src/app/api/dashboard/route.ts``
- ``src/app/api/missions/latest/route.ts``

Mission 005 should replace the runtime truth path with Neon-backed reads and writes while preserving the existing read contracts in ``src/lib/dashboard-types.ts``.